Event Data Recorder for Light Electric Vehicles (LEVs)

Hrishikesh Anand
Reg. No. 23BCT0124
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India

***Abstract*—Light electric vehicles (LEVs) such as electric scooters and e-bikes have become an important part of sustainable urban transport, but their lithium-ion battery packs can enter thermal runaway and release heat, flammable gases and fire. Battery management systems and telematics units detect abnormal conditions, yet in a severe battery event the same electronics, and the volatile data they hold, are often destroyed, so little record remains of the conditions that led to the failure. This paper presents a low-cost Internet of Things (IoT) Event Data Recorder (EDR) that changes how data is handled once signs of distress appear. An ESP32-WROOM-32 acquires temperature (DS18B20), a relative gas level (MQ-135-type module), a monitored voltage (resistive divider module) and motion (MPU6050). The firmware applies a 60 s gas warm-up, a learned gas baseline and persistence checks to seven rules (high temperature, high gas, rising gas, low voltage, high voltage, impact and free fall) and switches between NORMAL and CRITICAL states. Confirmed events trigger a buzzer and an LCD alert and enter a priority queue, which a separate FreeRTOS network task uploads to the Firebase Realtime Database over authenticated HTTPS ahead of routine telemetry. A web dashboard, VoltGuard, presents live values, history graphs and the event log. Bench tests demonstrated the complete sensor-to-dashboard path and the creation of separate event records. They also exposed a saturated gas channel, a simulated idle voltage value, and the absence of sensor calibration and vehicle-level validation.**

***Index Terms*—Event Data Recorder, Light Electric Vehicles, Internet of Things, ESP32, Battery Safety, Thermal Runaway, Firebase, Smart Cities, Electric Mobility.**

I. Introduction

Cities are adopting electric mobility to reduce local air pollution, noise and transport emissions. Light electric vehicles (LEVs) such as electric scooters, mopeds and e-bikes have low running costs, need little road and parking space, and suit commuting and last-mile deliveries in dense urban areas, where they complement public transport.

Almost every LEV stores its energy in a lithium-ion battery pack. Such packs can be abused through overcharging, external short circuits, mechanical damage or overheating. Under abuse, exothermic reactions inside a cell can generate heat faster than it is removed. The process then becomes self-sustaining. This condition, known as thermal runaway (TR), releases flammable and toxic gases and can propagate from one cell to its neighbours [1], [2]. Because LEVs are often parked and charged inside homes, shops, basements and delivery hubs, a battery fire in a city is a public-safety problem as much as a vehicle problem.

Production vehicles use a battery management system (BMS) to monitor cell voltage, current and temperature, often supplemented by a telematics unit. These systems are designed for normal and moderately abnormal operation. In a severe thermal event, however, the electronics that hold recent data sit near the heat source and are usually powered by the failing pack, so the latest records can be lost with the hardware. Post-incident analysis must then rely mainly on physical evidence from damaged cells [3], and even documented incidents are typically studied from the viewpoint of detection rather than data preservation [4].

The event data recorder is well established for road vehicles; in the United States, federal regulation specifies the data elements and survivability required of light-vehicle EDRs [5]. These recorders address crash dynamics, not battery precursors such as abnormal heating or gas release, and LEVs rarely carry any comparable low-cost recorder.

The motivation for this work is that the value of data depends on the state of the system. A conventional telemetry unit transmits at a fixed interval, so a sample taken just before a failure is treated like one taken during normal riding. If the device recognises that something unusual is happening, the data can be labelled as critical and sent first, while the link and the device still work. ESP32-class microcontrollers make this practical by combining Wi-Fi, a multi-channel ADC and local processing [6]–[8], and the literature suggests that early gas release, together with temperature, voltage and motion, is a useful indicator [9]–[12].

This paper describes the design, implementation and bench evaluation of an IoT-based EDR prototype for LEVs. Its contributions are: (i) a low-cost ESP32 acquisition unit with temperature, gas, voltage and motion channels and local alerts; (ii) firmware that confirms seven event rules using warm-up gating, baseline learning and persistence timing within a NORMAL/CRITICAL state model; (iii) a dual-core structure in which a priority event queue is uploaded to Firebase ahead of routine telemetry; (iv) the VoltGuard dashboard; and (v) an honest account of what the prototype has and has not demonstrated, together with its relevance to smart-city mobility.

The recorder is intended to complement, not replace, a certified BMS. Sections II and III review related work and state the gap and objectives; Sections IV and V present the architecture and implementation; Sections VI and VII report tests and results; Section VIII discusses smart-city applications and future work; and Section IX concludes.

II. Related Work and Literature Review

Sixteen sources published between 2021 and 2026 were reviewed, covering forensic studies, production detection hardware, low-cost monitoring prototypes, gas-based early warning and thermal-runaway warning reviews. Table I summarises each source and its relation to this work.

A. Forensic and Incident Studies

Yan, Marr and Ezekoye examined cylindrical cells and batteries that had already failed in order to identify characteristics that are useful to fire investigators [3]. Their work improves the interpretation of physical evidence, but it begins after the event, when electronic records are normally

gone. Gao *et al.* analysed a thermal-runaway incident in an electric vehicle and proposed an online method for detecting internal short circuits [4]. Neither study aims to keep event-time data available.

B. Production Detection Hardware

A granted patent assigned to Contemporary Amperex Technology describes a detection circuit in which sensing cables are routed close to the explosion-proof valves of the cells [13]. Resistor networks sample the cable state, and the battery management unit confirms thermal runaway when cable disruption coincides with pack-level conditions such as excessive temperature or a rapid temperature rise. The design is inexpensive and resistant to false alarms, but it responds only once venting has begun and produces an alarm rather than a record of preceding conditions.

C. Low-Cost Monitoring Prototypes

Usha Rani *et al.* built an ESP32-based battery monitor for electric vehicles that measures voltage and sends remote alerts [6]. Funde *et al.* implemented a microcontroller-based smart BMS with several sensors and local indication [7]. Kok *et al.* used ESP32-class hardware for an adaptive hazard-detection system that balances energy use, cost and detection capability [8]. These prototypes confirm that the platform is practical, but none of them changes the way data is prioritised or transmitted once a critical condition has been detected.

D. Gas-Based Early Warning

Off-gas detection has been evaluated for stationary storage [9] and demonstrated at laboratory scale [10]. Song *et al.* reviewed gas generation and compared semiconductor, electrochemical, catalytic and optical sensors, concluding that gases can precede major thermal or electrical anomalies but that selectivity, drift and false alarms remain open problems for low-cost sensors [11]. Metal-oxide nanomaterial sensors may improve sensitivity in future [14]. These sources justify a gas channel, while a generic MQ-series module gives only a relative indication.

E. Warning Frameworks and Reviews

Reviews have classified thermal-runaway warning technologies [1] and analysed triggers, mechanisms and early-warning strategies [2]. Han *et al.* compared temperature, gas, pressure and acoustic sensing for energy-storage systems [12]. Ou proposed an alarm framework for electric-vehicle battery packs [15], and Gu *et al.* derived a state-of-safety indicator that provides a long warning horizon at the cost of a model-based state estimator [16]. This work concentrates on prevention, with little attention to preserving evidence when prevention fails. Likewise, the regulatory EDR for road vehicles [5] targets crash dynamics rather than battery precursors.

TABLE I
COMPARATIVE SUMMARY OF THE REVIEWED LITERATURE

Source	Focus and approach	Limitation relative to this work
Yan *et al.* [3], 2021	Post-incident examination of failed cylindrical cells	Begins after the event; no pre-failure data captured
Gao *et al.* [4], 2021	EV thermal-runaway case with online internal short-circuit detection	Detection focus; data availability not addressed
Wang *et al.* [13], 2022	Patent: sensing cables near cell vents, BMU threshold confirmation	Triggers at venting; alarm only, no record stored or sent
Usha Rani *et al.* [6], 2023	ESP32 voltage sensing with remote alerts	Voltage only; no gas, temperature or motion
Funde *et al.* [7], 2024	Microcontroller-based smart BMS with local indication	No event-driven change in data handling
Purushothaman [9], 2024	Evaluation of off-gas detection techniques	Stationary storage, not LEVs
Hu *et al.* [1], 2024	Review and classification of TR warning methods	Prevention-oriented; evidence preservation not covered
Liu *et al.* [2], 2024	Review of TR triggers, mechanisms and warning strategies	Evidence preservation outside its scope
Wang *et al.* [10], 2024	Laboratory gas-sensor monitoring of TR	Laboratory setting, single parameter
Song *et al.* [11], 2025	Review of gas pathways and sensor families	Selectivity, drift and false alarms; low-cost sensors limited
Ou [15], 2025	Alarm framework for EV battery packs	Local alarm; no remote preservation
Han *et al.* [12], 2025	Review of temperature, gas, pressure and acoustic sensing	Energy-storage focus, not LEVs
Gu *et al.* [16], 2025	Unified state-of-safety indicator from battery data	Needs state estimation beyond a low-cost recorder
Kok *et al.* [8], 2025	Adaptive ESP32-class hazard monitoring	General hazards; not battery-specific
Qiu *et al.* [14], 2026	Review of MOS nanomaterial gas sensors	Mostly at materials stage; not yet deployable

III. RESEARCH GAP AND OBJECTIVES

A. Research Gap

Measuring temperature or voltage on an LEV is not new; the BMS and telematics hardware already do so. The gap identified in the reviewed sources concerns what happens to the data when a failure begins. Four aspects stand out:

1. *Lack of event-prioritised recording.* The reviewed prototypes and frameworks sample at a constant rate or raise an internal alert. When a critical condition is confirmed, the labelling, queuing and transmission of the data do not change.
2. *Loss of critical information during failure.* No mechanism keeps an event-time data stream available when the on-board electronics are damaged by the failing pack.
3. *Few independent low-cost solutions.* The publicly described systems are either built into the pack or are research instruments; an inexpensive add-on recorder is not described.
4. *No integrated multi-sensor event logging for LEVs.* None of the sources combines temperature, gas, voltage and motion detection with event logging for compact two-wheeler platforms.

These statements are limited to the sources examined in Table I. The project does not claim that no such system exists elsewhere.

B. Objectives

The project set the following objectives:

1. build a low-cost ESP32-based acquisition unit that measures temperature, relative gas concentration, a monitored voltage and motion;
2. detect events on the device using a warm-up stage, a tracked gas baseline and persistence checks, with two operating states, NORMAL and CRITICAL;
3. indicate the state locally using a 16×2 LCD and an active buzzer;
4. send periodic telemetry and a queue of critical events to the Firebase Realtime Database over secure HTTPS;
5. develop a web dashboard (VoltGuard) that shows live data, historical graphs and the critical-event log; and
6. design an enclosure and report the validation status and limitations of the prototype accurately.

IV. Proposed System and Architecture

A. Layered Architecture

The recorder is organised in five layers, as shown in Fig. 1. The sensing layer converts physical quantities into 1-Wire, analog and I^{2}C signals. The edge layer runs on the ESP32 and is divided between the two processor cores. Core 1 runs the sensing loop, which conditions readings, evaluates the event rules and drives the local outputs. Core 0 runs a network task that handles Wi-Fi, authentication and uploads. The two cores exchange data only through FreeRTOS queues, so a slow HTTPS request cannot delay a sensor reading or the alarm. The cloud layer consists of Firebase Authentication and the Firebase Realtime Database [19], [20]. The presentation layer is the VoltGuard dashboard, a static web application whose source is kept in a GitHub repository and deployed through Vercel.

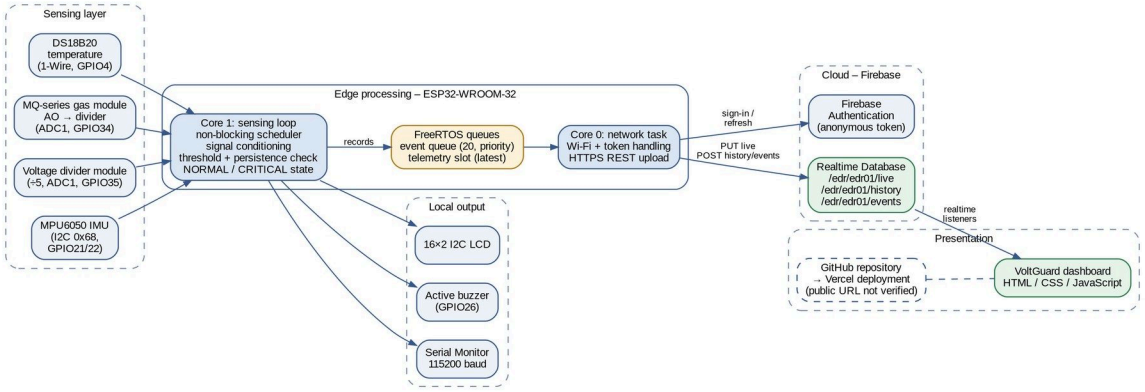


Fig. 1. System architecture of the Event Data Recorder. Sensing, edge processing on the two ESP32 cores, local output, the Firebase cloud layer and the VoltGuard presentation layer exchange data in the directions shown.

B. Hardware Components

Table II lists the hardware of the bench prototype and its pin mapping. The ESP32-WROOM-32 provides a dual-core Xtensa LX6 processor, 2.4 GHz Wi-Fi and a 12-bit successive-approximation ADC [17]. The DS18B20 measures temperature at the probe tip with 12-bit resolution [22]. The MQ-series module (MQ-135 type) responds to several gases and to humidity, and therefore gives only a relative indication after preheating and baseline calibration [21]. The voltage module is a passive divider intended for low-voltage sources. The MPU6050 provides a three-axis accelerometer and gyroscope [23]. A 16×2 character LCD with a PCF8574 I^{2}C backpack and an active buzzer driven through a transistor provide local feedback. The prototype is powered at 5 V from USB.

TABLE II
Hardware Components and ESP32 Pin Mapping

Component	Interface / ESP32 pin	Key specification and configuration	Role
ESP32-WROOM-32 DevKit	USB 5 V; ADC1, I^{2}C, GPIO	Dual-core Xtensa LX6, 2.4 GHz Wi-Fi, 12-bit SAR ADC, 3.3 V logic, 4 MB flash [17]	Acquisition, event logic, upload
DS18B20	1-Wire, GPIO4	−55 to +125 °C; ±0.5 °C from −10 to +85 °C; 0.0625 °C resolution [22]; 4.7 kΩ pull-up to 3.3 V	Temperature at the probe
MQ-series module (MQ-135 type)	AO → 2 × 4.7 kΩ divider → GPIO34 (ADC1_CH6)	5 V heater; relative response to several gases; needs preheating and baseline [21]	Relative gas indication
Voltage sensor module	S → GPIO35 (ADC1_CH7)	Passive 30 kΩ / 7.5 kΩ divider (÷5); input to be kept below about 15 V with the 3.3 V ADC	Monitored supply voltage
MPU6050 IMU	I^{2}C 0x68, GPIO21/22, 100 kHz	3-axis accelerometer and gyroscope [23]; set to ±8 g, ±500 °/s, ≈44 Hz DLPF	Impact, free fall, motion state
16×2 LCD + PCF8574	I^{2}C (shared bus)	Backpack address 0x27 or 0x3F detected at start-up; 5 V supply	Local readings and alerts
Active buzzer	GPIO26 via NPN transistor	Active-high drive	Audible critical alert

All modules share a single common ground. Analog inputs are on ADC1 because ADC2 is used by the Wi-Fi driver.

The pin assignment avoids the ESP32 strapping and flash pins. Both analog inputs are placed on ADC1, because ADC2 is used by the Wi-Fi driver and cannot be read reliably while Wi-Fi is active [17], [18]. The complete wiring is shown in Fig. 2.

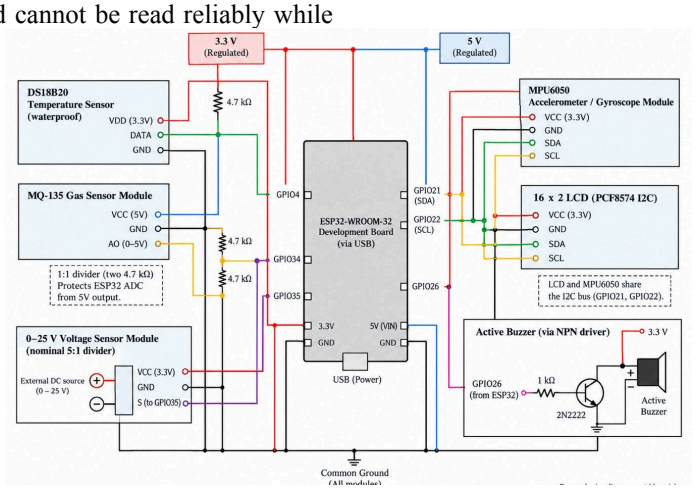


Fig. 2. Circuit diagram of the bench prototype. The MQ-135 output reaches GPIO34 through a 1:1 divider, the voltage module output goes to GPIO35, the MPU6050 and LCD share the I^{2}C bus on GPIO21/22, and the buzzer is switched by an NPN transistor from GPIO26.

The hardware imposes several constraints. The ESP32 inputs are not 5 V tolerant, so the MQ module output must be divided before it reaches the ADC. The voltage module was designed for 5 V boards and, with the ESP32, can measure only up to about 15 V; the 36 V and 48 V packs used in LEVs therefore cannot be measured directly. The DS18B20 reports the temperature at the probe location only, and the MQ module cannot identify a particular vented gas.

C. Software and Cloud Stack

The firmware was developed in Arduino IDE 2.3.10 with the Espressif ESP32 board package and is written in C++ on the Arduino framework, which runs on ESP-IDF and FreeRTOS. It uses the OneWire, DallasTemperature, LiquidCrystal_I2C and Wire libraries, a register-level MPU6050 driver written for the project, and the WiFi, WiFiClientSecure, HTTPClient and Preferences libraries of the ESP32 core. The dashboard uses HTML, CSS and JavaScript with the Firebase JavaScript SDK, is version-controlled with Git and GitHub, and is hosted on Vercel. Firebase Authentication with the anonymous provider issues an ID token to the device [19], and the Realtime Database stores all records [20]. The database rules allow reads and writes under `/edr/$device` only when `auth != null`. The database is located in the asia-southeast1 region.

D. Cloud Data Model

All data for one recorder is stored under `/edr/<DEVICE_ID>`, where the prototype uses `edr01`; the resulting tree is visible in the console capture of Fig. 5. The `live` node is overwritten with an HTTP PUT request and always holds the latest snapshot. The `history` and `events` nodes are appended with HTTP POST requests, which create children with unique push identifiers [20]. Each record carries a Firebase server timestamp, so the time reflects when the record reached the database rather than the board uptime. Each record contains `ts`, `uptime_ms`, `state` (NORMAL or CRITICAL), `temp_c` (null if the sensor is missing), `gas` (averaged raw ADC counts, 0–4095), `volt_v`, `gyro` (MOVING, STILL or "--") and `acc_g`; event records add `event_no`.

V. Methodology and Implementation

A. Hardware Assembly and Sensor Interfacing

The prototype was assembled on a breadboard. The development board supplied a 5 V rail from its VIN pin and a 3.3 V rail from its regulator. The DS18B20 was powered at 3.3 V with a 4.7 kΩ pull-up on its data line [22]. The MQ module heater required 5 V [21], and because its output can approach that value, it was connected to GPIO34 through two 4.7 kΩ resistors that halve the voltage. A blank LCD, caused by powering its backpack from 3.3 V, was fixed by using 5 V and adjusting the contrast, and failed uploads succeeded once the peripherals were disconnected and the IO0 button was held at 115200 baud.

B. ESP32 Initialisation

The `setup()` function opens the serial port at 115200 baud, selects 12-bit ADC resolution with 11 dB attenuation, starts the I^2C bus at 100 kHz and scans it for the IMU and LCD. Each sensor is initialised and reported as OK or "FAILED – will retry", so a missing module does not stop the others. Finally, the buzzer beeps, the event queue (20 records) and single-slot telemetry queue are created, and the network task is started on core 0.

C. Sensor Data Acquisition

Rather than reading every sensor in one pass and then waiting, the firmware gives each task its own `millis()`-based timer and returns immediately when a task is not due. This cooperative, non-blocking scheduler keeps the loop short and lets fast and slow signals coexist: the MPU6050 is read every 20 ms (a 14-byte burst from register 0x3B), the analog channels every 100 ms, the DS18B20 every 800 ms, an I^2C health check runs every 1 s with a 3 s retry, the LCD refreshes every 500 ms, a serial report is printed every 1 s, and a telemetry snapshot is taken every 5 s or immediately on a state change.

Temperature: The DS18B20 runs at 12-bit resolution in non-blocking mode. Every 800 ms the firmware collects the finished conversion and requests the next one. The 85 °C value that the sensor reports before its first conversion is discarded, and three consecutive "disconnected" codes mark the sensor as missing, after which it is retried every 3 s.

Gas and voltage (filtering and averaging): Both analog channels are read every 100 ms. Each reading is the mean of $N = 8$ samples of the raw count and of the calibrated pin voltage returned by `analogReadMilliVolts()`, which applies the factory calibration of the chip [18]:

$$V_{\text{pin}} = (1/N)\,\Sigma_{i=1\ldots N}\,V_{\text{pin},i}, \quad N = 8 \tag{1}$$

The monitored input voltage is recovered from the divider ratio k, which is 5 for the 30 kΩ/7.5 kΩ module, multiplied by a calibration factor that is currently 1.00:

$$V_{\text{in}} = k \cdot V_{\text{pin}}, \quad k = (R_1 + R_2)\,/\,R_2 \tag{2}$$

If the raw count is below 20 or V_{in} is below 0.2 V, no source is considered connected. In that case the current firmware substitutes a slowly varying value between 0.10 V and 0.40 V so that the display is not static. This behaviour is controlled by the constant `SIMULATE_IDLE_VOLTAGE` and is discussed in Section VII.

Motion: The MPU6050 is driven by a register-level routine because the available library rejected compatible clone chips. The routine sets a 100 Hz sample rate, a digital low-pass filter of about 44 Hz, a ±500 °/s gyroscope range and a ±8 g accelerometer range [23]. At start-up it takes $M = 100$ samples at rest to estimate the gyroscope bias and the accelerometer sensitivity S in counts per g. This calibration was added after the module read 0.66 g at rest with the nominal scale factor:

$$S = (1/M)\,\Sigma_{j=1\ldots M}\,\sqrt{(c_x^2 + c_y^2 + c_z^2)}, \quad M = 100 \tag{3}$$

$$|a| = \sqrt{(a_x^2 + a_y^2 + a_z^2)}\,/\,g \tag{4}$$

Here c denotes raw accelerometer counts and g is standard gravity. The board is reported as MOVING when the angular-rate magnitude exceeds 8 °/s or when $|a|$ differs from 1 g by more than 0.15 g, and as STILL after 1 s without such motion.

D. Gas Baseline Learning and Warm-Up

An MQ-series sensor gives a relative value that varies with temperature and humidity. The gas rules therefore compare the averaged reading G both with an absolute threshold and with a learned baseline B, which is updated as an exponential moving average:

$$B \leftarrow B + \alpha\,(G - B) \tag{5}$$

During the first 60 s, while the heater stabilises, $\alpha = 0.05$ so that the baseline settles quickly and both gas rules are disabled. After warm-up, $\alpha = 0.001$, and the baseline is frozen while a gas rule is active, so that a genuine rise is not absorbed into the baseline.

E. Threshold-Based Event Detection

Seven rules are evaluated on every pass of the loop (Table III). A rule is confirmed only when its condition has remained true for the persistence time t_{p}; if the condition becomes false, its timer resets. Impacts are confirmed immediately because they are short. The low-voltage rule is active only when $V_{\text{in}} \geq 1.0$ V, which avoids an alarm when no source is connected. The thresholds are example values chosen for bench testing. They are *not* validated battery or vehicle safety limits.

TABLE III
EVENT RULES IMPLEMENTED IN THE FIRMWARE (PROTOTYPE BENCH-TESTING THRESHOLDS)

Code	Condition	t_p	Gating
`TEMP_HIGH`	$T \geq 45$ °C	1.5 s	Valid DS18B20 reading
`GAS_HIGH`	$G \geq 3000$ counts	1.5 s	After 60 s warm-up
`GAS_RISE`	$G - B \geq 400$ counts	1.5 s	After 60 s warm-up
`VOLT_LOW`	$V_{in} < 10.5$ V	1.5 s	Only if $V_{in} \geq 1.0$ V
`VOLT_HIGH`	$V_{in} > 14.8$ V	1.5 s	—
`ACC_IMPACT`	$\|a\| \geq 2.5$ g	0	Valid IMU data
`FREEFALL`	$\|a\| \leq 0.30$ g	150 ms	Valid IMU data

The voltage limits correspond to the low-voltage bench range of the current module, not to a 36 V or 48 V LEV pack.

F. NORMAL and CRITICAL Operating States

The function `evaluateEvents()` combines the confirmed rules into a bit mask of active reasons. Fig. 3 shows the resulting state machine. After `setup()`, the recorder spends 60 s in warm-up with the gas rules disabled. When any rule is confirmed in the NORMAL state, the recorder enters CRITICAL, shows the LCD alert page, increments the event counter, logs the record to the serial monitor, queues an event record for the cloud and forces an immediate telemetry update. While CRITICAL, a rule that is not yet part of the current episode produces another numbered event; a rule that is already part of the episode does not create a duplicate. When no rule has been active for 5 s, the episode ends, the recorder returns to NORMAL and a further immediate telemetry update is forced so that the dashboard reflects the change at once.

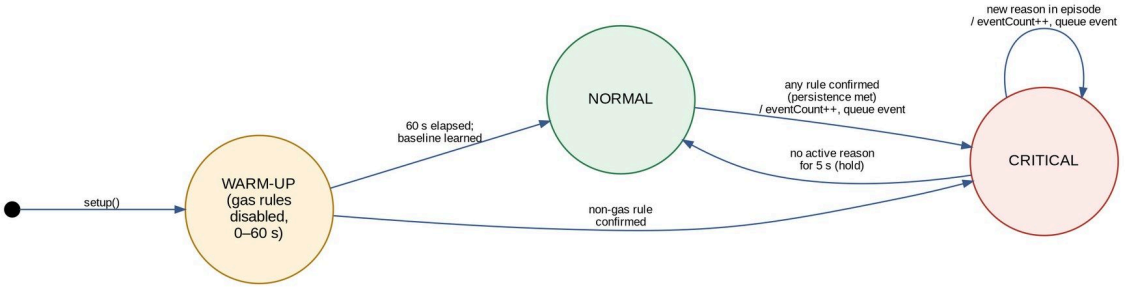


Fig. 3. Operating-state diagram of the recorder. A confirmed rule moves the recorder from NORMAL (or, for non-gas rules, from WARM-UP) to CRITICAL; a 5 s hold with no active reason returns it to NORMAL.

G. Priority-Based Event Queue

Two FreeRTOS queues connect the cores. The event queue holds up to 20 event records in RAM. The telemetry queue has a single slot, which is overwritten by the newest snapshot every 5 s or immediately on a state change. The network task always checks the event queue first. It reads an event with `xQueuePeek()`, uploads it and removes it with `xQueueReceive()` only after the database returns HTTP 200. Routine telemetry is sent only when the event queue is empty. In this way an event is never displaced by routine data, and an unsent event remains queued for retry.

H. LCD and Buzzer Alerts

In NORMAL, the LCD alternates every 2 s between two pages: temperature and gas, then voltage and motion state. In CRITICAL, an alert page reading "!! CRITICAL !!" with short reason codes (for example GAS, TMP or IMP) is inserted at every other page change, and the display is refreshed every 500 ms. The buzzer is silent in NORMAL and alternates 250 ms on and 250 ms off in CRITICAL, using the same non-blocking timing as the rest of the loop. Because these outputs are driven from core 1, they do not depend on network availability.

I. Firebase Authentication and Data Transmission

All network operations run in `networkTask()`, which executes on core 0 with a 12 KB stack. At start, the task restores the device refresh token from non-volatile storage so that the board keeps the same anonymous identity across reboots. In its main loop it first ensures Wi-Fi, calling `WiFi.begin()` at most every 10 s. It then ensures a valid ID token: an expired token is refreshed through the secure-token endpoint, and, as a fallback, a new anonymous user is created through the sign-up endpoint and its refresh token is stored in flash [19]. Event records are serialised to JSON and posted to `/edr/edr01/events`. A failed upload is retried after 2 s, and an HTTP 401 response clears the token so that it is refreshed. Otherwise, telemetry is written with PUT to `/live`; every twelfth snapshot (about once per minute) and every snapshot taken while CRITICAL is also posted to `/history`.

JSON is built with `snprintf()` in a static 384-byte buffer. As prototype trade-offs, the HTTPS client does not validate the server certificate (`setInsecure()`) and the Wi-Fi credentials are compiled into the firmware; a production version should validate certificates, provision credentials at run time and enable flash encryption.

J. VoltGuard Dashboard

VoltGuard is a single-page web application built with HTML, CSS and JavaScript using the Firebase JavaScript SDK, without a build step. Because the database rules reject unauthenticated reads, the page signs in with Firebase Authentication before attaching real-time listeners. It provides live cards for temperature, gas, voltage and acceleration from `/live`; a status band showing NORMAL or CRITICAL; an SVG history graph from `/history` with a selector for the plotted quantity; and a table of critical events from `/events` with an event count. The source is kept in a GitHub repository and deployed on Vercel.

K. Historical Data and Event Logging

The `live` node shows the current state and never grows. The `history` node keeps about one entry per minute in NORMAL and one per 5 s in CRITICAL, so more detail is kept when it matters most. The `events` node stores one entry per numbered event and is the record the recorder is designed to preserve. Because the reason field was removed from cloud records during payload optimisation, the dashboard shows that an event occurred and its readings, but not which rule triggered it.

VI. EXPERIMENTAL SETUP AND TESTING

All tests were carried out on the bench. The prototype was powered from a laptop USB port and connected to a 2.4 GHz mobile hotspot, with the VoltGuard page open in a browser and the Firebase console used to inspect records. No test was performed on an operating vehicle, with a real battery pack, or during an actual or induced thermal event.

Tables IV and V list the test cases with their observed outputs and status; the expected outputs follow from the rules and intervals defined in Section V. "Pass" indicates that supporting evidence exists, "Fail" that a fault was observed, and "Pending" that the test has not yet been completed.

TABLE IV
HARDWARE AND FIRMWARE TEST CASES

ID	Test	Observed	Status
HT01	Compile for "ESP32 Dev Module"	Compiled; about 81% of program storage used	Pass
HT02	Upload firmware over USB	Succeeded with peripherals disconnected, 115200 baud and IO0 held	Pass (workaround)
HT03	I^2C scan at power-on	IMU detected; LCD address result to be recorded	Pending
HT04	DS18B20 at room temperature	22.4 °C displayed	Pass (accuracy not verified)
HT05	MPU6050 at rest after calibration	0.66 g before scale calibration; 1.00 g after	Pass
HT06	Gas channel in clean air after warm-up	3844 counts (Firebase) and 4095 (dashboard); GAS_HIGH raised	Fail (divider wiring)
HT07	Voltage channel, no source	0.26 V displayed; no voltage alarm	Pass (simulated value)
HT08	Voltage channel with 9–12 V bench supply	Not yet performed	Pending
HT09	LCD pages in NORMAL and CRITICAL	Blank display traced to 3.3 V supply and contrast; corrected	Pass
HT10	Tap the IMU sharply	MOVING shown; critical event recorded	Pass
HT11	Warm DS18B20 above 45 °C for >1.5 s	To be recorded	Pending

Fault handling was exercised through the start-up scan, per-sensor reporting and the I^2C health check (HT03, HT09). Wi-Fi outage recovery (ST-10) was not performed, so the queue-and-retry logic of Section V-G is implemented but not yet verified.

TABLE V
CLOUD AND DASHBOARD TEST CASES

ID	Test	Observed	Status
ST-01	Host test of `buildJson()`	Event and telemetry outputs parsed with exactly the expected fields	Pass
ST-02	Connect to 2.4 GHz hotspot	Board reached Firebase (data written)	Pass
ST-03	Anonymous sign-in, auth-only rules	Records written under `/edr/edr01`	Pass
ST-04	Observe `/live` for one minute	Populated with current readings; interval not yet timed	Pass (interval untimed)
ST-05	Trigger critical events	Event records created; count of 16 shown	Pass (false positives)
ST-06	NORMAL for 5 min	To be recorded	Pending
ST-07	Open VoltGuard (local hosting)	22.4 °C, 4095, 0.26 V, 1.00 g displayed	Pass
ST-08	History graph and event table	Event count displayed; screenshots to be captured	Pending
ST-09	Read database without sign-in	Not yet performed	Pending
ST-10	Disable Wi-Fi, trigger event, restore	Not yet performed	Pending
ST-11	Open Vercel deployment	Public URL verified	Pass

VII. RESULTS AND DISCUSSION

A. End-to-End Operation

The assembled bench prototype is shown in Fig. 4. Readings were acquired on the ESP32, written to Firebase under authentication-only rules and displayed by VoltGuard, and critical events were stored as separate records. The LCD in Fig. 4 shows the alert page raised by a gas event.

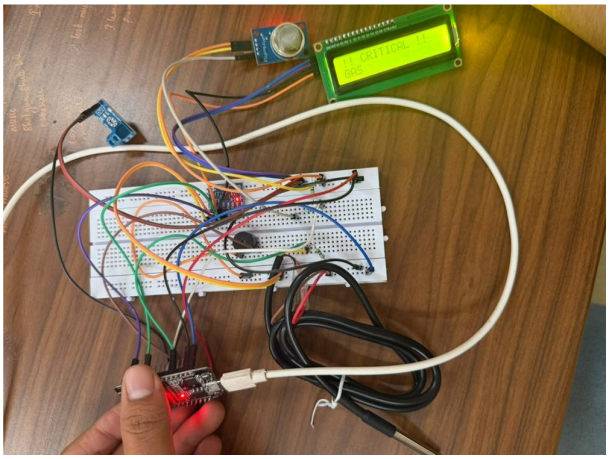


Fig. 4. Complete bench prototype on a breadboard, powered over USB. The LCD shows a CRITICAL alert page.

Fig. 5 shows the Firebase console with the `edr01` node expanded. The `live` child holds the record fields listed in Section IV-D, including the server timestamp, and the `history` and `events` children were populated during operation. The VoltGuard dashboard (Fig. 6) presents the same values in live cards, together with a status band, the history graph and device diagnostics.

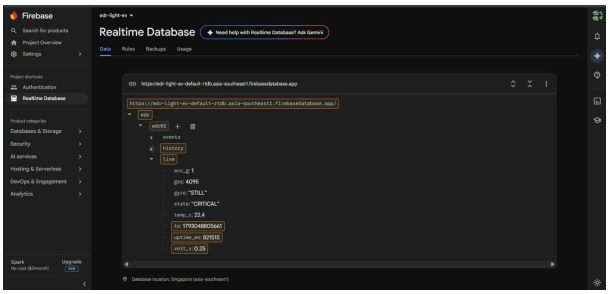


Fig. 5. Firebase Realtime Database console showing the `/edr/edr01` tree and a `live` record in the CRITICAL state.

B. Observed Values

One dashboard screenshot showed 22.4 °C, a gas reading of 4095 counts, 0.26 V, 1.00 g and a count of 16 critical events. These are single, uncalibrated observations; the temperature and acceleration are plausible, while the gas and voltage values are interpreted below.

C. Gas Sensor Saturation

A reading of 4095 means that the pin was at or very near the top of the ADC range. With 11 dB attenuation, producing this through the 1:1 divider would require a module output well above its 5 V supply, which is not possible. The divider was therefore not working as intended; either the signal reached the pin directly or the divider was wired incorrectly. The gas readings are consequently invalid, and the condition may also exceed the input limits of the pin [17]. Because the value was above the 3000-count GAS_HIGH threshold, the recorder correctly applied the rule, so the event-logging path itself

behaved as designed. The fix is to verify the module and pin voltages with a multimeter, rebuild the divider, relearn the baseline and repeat HT06.

D. Voltage Measurement Limitations

Below the presence threshold, the firmware substitutes a drifting value between 0.10 V and 0.40 V; the observed 0.26 V lies in this band and no source was connected, so it was simulated. Because the simulation flag was removed from the cloud payload, stored records cannot show whether a voltage was measured or simulated. The simulation should be disabled or the flag restored before checking the channel against a bench supply and multimeter (HT08); even then, the module covers only low-voltage sources.

E. Temperature, Motion and Historical Data

The temperature and acceleration readings indicate that the DS18B20 driver and the IMU calibration work. The calibration assumes that the board is stationary just after power-up; moving it at that moment would bias the scale factor. Fig. 6 shows the history graph for temperature drawn from the `/history` node, confirming that historical records were stored and retrieved for visualisation.

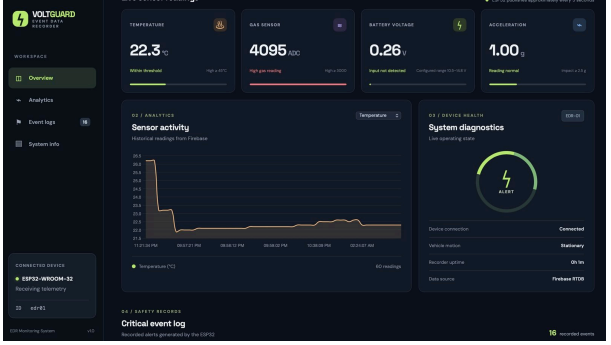


Fig. 6. VoltGuard dashboard: live cards, historical temperature graph, plotted from `/history` records, with the device diagnostics panel and event count.

F. Critical Event Recording

Fig. 7 shows the event table, which listed the 14 most recent records at the time of capture; the screenshot discussed in Section VII-B showed a count of 16. Every listed record has a gas value of at least 3000 counts, which is consistent with the saturated channel triggering GAS_HIGH. Five records also store an acceleration magnitude at or above the 2.5 g impact threshold (2.57, 2.64, 3.07, 4.78 and 5.50 g), which is consistent with the tap tests of HT10. Without stored reason codes these attributions cannot be confirmed per record. Event numbers restart from #1 on several dates because the counter is held in RAM and resets on reboot. Most stored events are therefore best treated as false positives caused by the gas fault, although they confirm that the path from rule confirmation to the dashboard table worked.



Fig. 7. Critical-event log in VoltGuard, read from `/events`. Each row is one stored event record with the readings captured at the time of the event.

G. Demonstrated Features and Remaining Validation

In summary, the prototype demonstrated multi-sensor acquisition (with the gas channel needing rewiring and the voltage channel untested with a source), on-device event detection with persistence, warm-up and state logic, local LCD and buzzer alerts, authenticated and prioritised upload of live, history and event records, and a working VoltGuard dashboard with a verified Vercel deployment. Wi-Fi outage recovery, unauthenticated-access denial, sensor calibration and the enclosure (designed in CAD but not fabricated) still require validation.

Beyond the sensor faults above, the recorder has not been installed on an LEV or tested during a real or simulated thermal-runaway event. Events are held only in RAM and there is no independent backup power, so at present the recorder would share the fate of the vehicle electronics. Uploading also requires a 2.4 GHz Wi-Fi network, and security is simplified as described in Section V-I.

VIII. Smart City Applications and Future Scope

The applications below are possibilities that follow from the architecture; they have not been implemented or tested.

A. Relevance to Smart and Sustainable Mobility

Electric micromobility supports sustainable urban development by moving short trips away from combustion vehicles, but it depends on public confidence in battery safety, especially where vehicles are charged indoors. The original project linked this work to Sustainable Development Goals 9, 11 and 12. By preserving the data needed to explain battery incidents, the recorder supports evidence-based improvements to vehicles, charging practice and urban safety rules.

Architecturally, the recorder is a small smart-city IoT pipeline: edge devices sense and decide locally, a cloud database aggregates records per device, and a web application presents them, while the safety-critical local alarm stays independent of the network.

B. Potential Applications

Urban EV fleet monitoring: Because each recorder writes under `/edr/<DEVICE_ID>`, the data model already supports multiple units, so a fleet dashboard for shared-scooter or delivery operators could list vehicles in the CRITICAL state with their recent history.

Battery safety monitoring and service centres: Pre-event records of temperature, gas and voltage could help service centres decide whether a pack should be inspected or withdrawn.

Post-incident investigation: Server-timestamped event records stored off the vehicle could support root-cause analysis and insurance or regulatory investigation, complementing the physical evidence studied in [3]. This would require calibrated sensors and validated thresholds first.

Transportation safety analysis: Aggregated impact and free-fall events from many vehicles could, in principle, inform road-safety studies or link with charging hubs. The prototype collects no location or personal data; a field version would need a clear data-ownership and consent policy.

C. Future Improvements

The following improvements are planned and are not part of the current implementation:

1. *Independent backup power:* the 18650 cell, protected TP4056 charger and boost converter modelled in enclosure Configuration B, so the recorder keeps transmitting after vehicle power is lost.
2. *Local non-volatile storage:* flash (e.g. LittleFS) or microSD storage of recent readings and all events, uploaded when connectivity returns.
3. *Improved calibration:* a corrected gas divider, characterised MQ baseline and drift, reference calibration of temperature and voltage, and thresholds from battery manufacturer data.
4. *Pack-voltage measurement:* a divider of about 23:1 with fusing and surge protection for 36 V and 48 V packs.
5. *Vehicle-level testing* while riding and charging, under real vibration, temperature and connectivity conditions.
6. *Rugged enclosure:* PETG or ABS printing with cable glands and a vent membrane, tested for vibration and splash.
7. *Communication reliability and security:* cellular or LoRa fallback, reason codes in event records, certificate validation, run-time credentials and per-device database rules.

IX. Conclusion

This paper addressed the loss of event-time data when a lithium-ion battery fails in a light electric vehicle. The proposed IoT Event Data Recorder uses an ESP32-WROOM-32 to acquire temperature, relative gas, voltage and motion data, confirms seven rules through warm-up gating, baseline learning and persistence timing, and switches between NORMAL and CRITICAL states. Splitting the work between the two cores, with a priority event queue and a single-slot telemetry queue, keeps local alerting independent of the network and uploads event records to Firebase ahead of routine data. VoltGuard presents live values, history and the event log.

Bench tests demonstrated the complete sensor-to-dashboard path, authenticated cloud storage, separate event records and local alerts. They also revealed a gas channel saturated by a wiring fault, a simulated idle voltage, and pending tests including Wi-Fi outage recovery. The recorder has not been tested on a vehicle or during a battery failure and should be regarded as a proof of concept that complements, rather than replaces, a vehicle's BMS. Within these limits, the work shows that event-prioritised recording for LEVs can be built from low-cost components and fits naturally into the IoT layers of smart, connected urban mobility. Backup power, local non-volatile storage, calibration, pack-voltage measurement and vehicle-level validation are the next steps towards a field-ready device.

Acknowledgment

The author thanks Prof. Sharmila Banu K, School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, under whose supervision the prototype described in this paper was originally developed as a B.Tech. Project-I, and the School for the laboratory resources used in its construction and testing.

References

[1] D. Hu *et al.*, "A review on thermal runaway warning technology for lithium-ion batteries," *Renew. Sustain. Energy Rev.*, vol. 206, 2024.

[2] C. Liu *et al.*, "Review—Understanding thermal runaway in lithium-ion batteries: Trigger, mechanism, and early warning strategies," *J. Electrochem. Soc.*, vol. 171, no. 12, Art. no. 120527, 2024.

[3] H. Yan, K. C. Marr, and O. A. Ezekoye, "Towards fire forensic characteristics of failed cylindrical format lithium-ion cells and batteries," *Fire Technol.*, vol. 57, pp. 1723–1752, 2021.

[4] W. Gao *et al.*, "Case study of an electric vehicle battery thermal runaway and online internal short-circuit detection," *IEEE Trans. Power Electron.*, vol. 36, no. 3, pp. 2452–2455, 2021.

[5] U.S. National Highway Traffic Safety Administration, "Event data recorders," *Code of Federal Regulations*, Title 49, Part 563.

[6] R. Usha Rani, G. Divya, M. Lakshmi Swarupa, and K. Navaneetha, "IoT based battery monitoring system for electric vehicle using ESP32," in *Proc. IEEE Int. Conf. Smart Technol. Comput., Elect. Electron. (SmartTechCon)*, 2023.

[7] V. Funde, M. Gurpude, and S. Latelwar, "Design and implementation of microcontroller based smart battery management system," in *Proc. 2nd IEEE Int. Conf. Smart Technol. Power Renew. Energy (SPECon)*, 2024.

[8] C. L. Kok, J. B. Heng, Y. Y. Koh, and T. H. Teo, "Energy-, cost-, and resource-efficient IoT hazard detection system with adaptive monitoring," *Sensors*, vol. 25, no. 6, Art. no. 1761, 2025.

[9] S. Purushothaman, "Evaluation of off-gas detection in Li-ion battery energy storage systems," in *Proc. IEEE Elect. Energy Storage Appl. Technol. Conf. (EESAT)*, 2024.

[10] X.-X. Wang *et al.*, "Monitoring thermal runaway of lithium-ion batteries by means of gas sensors," *Sens. Actuators B, Chem.*, vol. 411, 2024.

[11] Y. Song, X. Jiang, N. Lyu, H. Lu, D. Zhang, H. Li, and Y. Jin, "Early warning of lithium-ion battery thermal runaway based on gas sensors," *eTransportation*, vol. 26, Art. no. 100502, Dec. 2025, doi: 10.1016/j.etran.2025.100502.

[12] Y. Han *et al.*, "Advances in early warning of thermal runaway in lithium-ion battery energy storage systems," *Adv. Sensor Res.*, 2025.

[13] X. Wang, Y. Li, Z. Zeng, and W. Zhang, "Thermal runaway detection circuit and method," U.S. Patent 11 353 515 B2, Jun. 7, 2022.

[14] W. Qiu *et al.*, "The role of gas sensors based on metal oxide semiconductor nanomaterials in thermal runaway early warning for battery safety: A review," *ACS Appl. Nano Mater.*, vol. 9, 2026.

[15] T.-C. Ou, "Research on thermal runaway alarm framework for lithium-ion batteries of electric vehicles," in *Proc. IEEE Ind. Appl. Soc. Annu. Meeting (IAS)*, 2025.

[16] X. Gu *et al.*, "Early warning of thermal runaway based on state of safety for lithium-ion batteries," *Commun. Eng.*, vol. 4, Art. no. 106, 2025.

[17] Espressif Systems, "ESP32 series datasheet," ver. 5.3. [Online]. Available: https://www.espressif.com/documentation/esp32_datasheet_en.pdf

[18] Espressif Systems, "ESP-IDF programming guide: Analog to digital converter." [Online]. Available: https://docs.espressif.com/projects/esp-idf/ (accessed Oct. 2026).

[19] Google LLC, "Firebase Authentication REST API." [Online]. Available: https://firebase.google.com/docs/reference/rest/auth (accessed Oct. 2026).

[20] Google LLC, "Firebase Realtime Database documentation." [Online]. Available: https://firebase.google.com/docs/database (accessed Oct. 2026).

[21] "MQ-135 gas sensor technical data," datasheet (hosted by Olimex). [Online]. Available: https://www.olimex.com/Products/Components/Sensors/Gas/SNS-MQ135/resources/SNS-MQ135.pdf

[22] Maxim Integrated (Analog Devices), "DS18B20 programmable resolution 1-Wire digital thermometer," datasheet, Rev. 6, Jul. 2019. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf

[23] InvenSense Inc., "MPU-6000 and MPU-6050 product specification," Rev. 3.4, 2013.